

Title

Subtitle



Full name

Prepared for MONTH DAY-NUMBER, YEAR

Contents

1	Introduction	3
2	Methodology and Algorithms	4
2.1	Method 1: Particle Swarm Optimisation (PSO)	4
2.1.1	Multi-objective algorithm	4
2.2	Method 2: Simulated Annealing (SA)	5
3	Problem 1: Two-Bar Plane Truss Design	8
3.1	Problem description	8
3.2	Methodology	9
3.2.1	General explanation	9
3.2.2	Penalty function	9
3.3	Results	10
3.3.1	PSO simulation	10
3.3.2	SA simulation	13
3.4	Discussion	16
4	Problem 2: Sensor Calibration	17
4.1	Problem description	17
4.2	Cost function	18
4.3	Methodology	18
4.4	Settings	18
4.5	Result	19
4.6	Work discussion	20
5	Problem 3: Gear Train Design	21
5.1	Problem description	21
5.2	Cost function	22
5.3	Sources	22
5.4	Methodology	22
5.5	Settings	23
5.6	Results	24

5.6.1	Results analysis	24
5.6.2	Explanations	25
5.6.3	Conclusion	26
6	General Conclusion	27
6.1	Performance Comparison	27
6.2	Hyperparameter Sensitivity and Robustness	27
7	References	30
8	Appendix: Python Code	31
8.1	Two-Bar Plane Truss Problem Definition	31
8.2	PSO Simulation Script (Truss)	37
8.3	Steinhart-Hart Problem Definition	56
8.4	Simulated Annealing Class	62
8.5	Gear Train Problem and Runner	71

1 Introduction

This report investigates unconstrained and constrained nonlinear optimisation techniques. Specifically, it compares the performance of **Particle Swarm Optimisation (PSO)** and **Simulated Annealing (SA)**. These algorithms are applied to three distinct engineering problems:

1. **Two-Bar Plane Truss Design** (Constrained, Multi-objective)
2. **Sensor Calibration** (Unconstrained, Curve fitting)
3. **Gear Train Design** (Constrained, Discrete/Integer optimization)

2 Methodology and Algorithms

2.1 Method 1: Particle Swarm Optimisation (PSO)

The Particle Swarm Optimisation (PSO) is a stochastic optimisation technique inspired by the social behaviour of birds. A population of candidate solutions moves through the domain. Through several time steps, each particle adjusts its position based on its personal best and on the global best. This allows the algorithm to converge to a solution that matches a criterion [1].

2.1.1 Multi-objective algorithm

Description

The multi-objective PSO finds Pareto-optimal solutions using a selection based on dominance and diversity [2]. It presents advantages such as an ease of implementation, few parameters to tune, and a good application for nonlinear problems; but also, limitations such as a possible too early convergence, a sensitivity to parameters settings, and no guarantee of global optimum [1].

The MOPSO-CD (Multi-Objective Particle Swarm Optimisation with Crowding Distance) is an advanced extension of the PSO [3]. This particular algorithm was chosen from a list of other solutions on the base of specific criteria (Appendix A). Its key features are:

- Crowding distance-based leader selection,
- External archive management,
- Tournament selection,
- Enhanced exploration.

Performance metrics

Metrics are used to measure the performance of the algorithm. We have selected some of the most common criteria:

- The best value per objective which tracks the minimum value achieved for each objective function at every generation,
- The Pareto front size evolution which tracks how many solutions are on the Pareto front at each generation,
- The normalized convergence which compares both objectives normalized to $[0, 1]$.

2.2 Method 2: Simulated Annealing (SA)

Problem description:

The Simulated Annealing is an optimization method inspired by the annealing process in metallurgy. The algorithm explores the search space and accepts neighbouring solutions. In this algorithm, ameliorations are always accepted, and degradations are accepted with a probability. The probability decreases step after step (decreasing temperature). At the beginning, this acceptance mechanism helps to escape local minima. Afterwards, at the end of the process, the algorithm converges to a global minimum.

Cost function:

On this method we use this function to decide if a worst solution can be accepted:

$$P(\text{acceptance}) = e^{-\frac{E_{\text{new}} - E_{\text{actual}}}{T}} \quad (1)$$

Where:

- E_{actual} : Actual energy
- E_{new} : The new worst energy
- $P(\text{acceptance})$: the probability to accept E_{new}
- T : temperature parameter

Every iteration the temperature T decreases by the cooling_rate:

$$T = T \times \text{cooling_rate} \quad (2)$$

Multi-Objective Strategy (Weight Scalarization):

Unlike MOPSO, the Simulated Annealing program is inherently a mono-objective algorithm. To solve the bi-objective Truss problem, a scalarization approach is used to transform the problem into a single objective function.

- **Scalarization:** A weighted sum combines the two objectives (Weight and Displacement) into a single scalar Energy function.
- **Normalization:** A scaling factor (Scale = 10000) is applied to the displacement term to normalize both objectives to comparable orders of magnitude.
- **Weight Distribution:** To approximate a Pareto front, 200 independent runs are performed with the weight w_1 uniformly distributed between 0.005 and 0.995 (where $w_2 = 1 - w_1$). This allows the algorithm to explore different trade-offs, from minimizing weight ($w_1 \approx 1$) to minimizing displacement ($w_1 \approx 0$).

Performance metrics:

To evaluate the performance of the SA algorithm, the following metrics were tracked:

- **Convergence profiles:** Analysis of the best energy and current energy evolution over iterations to assess stability and exploration.
- **Pareto Front Approximation:** The collection of non-dominated solutions found across the 200 independent runs is plotted to visualize the trade-off curve (the "Banana shape").
- **Decision Space Analysis:** Mapping the decision variables (x_1, x_2) of the final solutions to identify distinct regions corresponding to different trade-off preferences.

Settings: It is important to consider:

- *initial_temp*: the temperature at the beginning
- *cooling_rate*: The speed of the freezing
- *min_temp*: The temperature where we stop the process

3 Problem 1: Two-Bar Plane Truss Design

3.1 Problem description

The Two-Bar Truss problem is composed by two diagonal bars (of aluminium here) connected on a node and are fixed at two upper point (Figure 1). On the central node we applied a vertical strength. This force represents the external load that the structure must support. We look for the vertical displacement of the node.

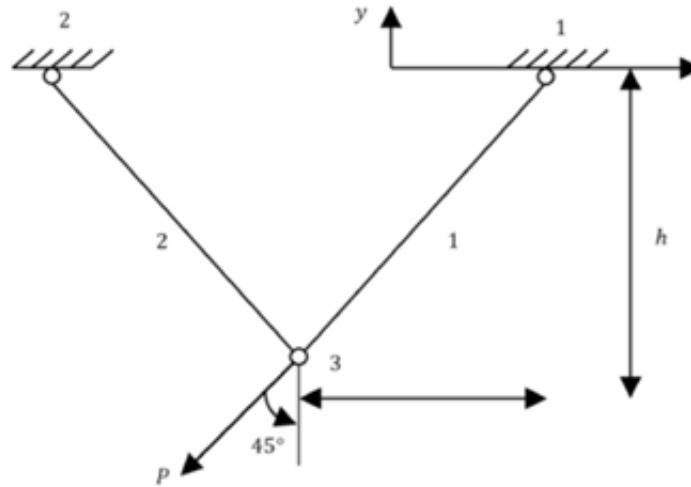


Figure 1: Two-bar plane truss

There are two design variables which are the position of the joints 1 and 2. They are defined by the two equations: f_1 the structural weight truss, and f_2 the displacement of joint 3.

$$f_1(x) = 2\rho h x_2 \sqrt{(1 + x_1^2)} \quad (3)$$

$$f_2(x) = \frac{Ph(1 + x_1^2)^{1.5} \sqrt{(1 + x_1^4)}}{2\sqrt{2}Ex_1^2x_2} \quad (4)$$

These functions are constrained by two stress constraints functions g_1 (tension) and g_2

(compression).

$$g_1(x) = \frac{P(1+x_1)\sqrt{1+x_1^2}}{2\sqrt{2}x_1x_2} - \sigma_0 \leq 0 \quad (5)$$

$$g_2(x) = \frac{P(-x_1+1)\sqrt{1+x_1^2}}{2\sqrt{2}x_1x_2} - \sigma_0 \leq 0 \quad (6)$$

The variables x_1 and x_2 are calculated from the following formulas: $x_1 = \frac{x}{h}$ and $x_2 = \frac{A}{A_{min}}$
 $0.1 \leq x_1 \leq 2.25$ and $0.5 \leq x_2 \leq 2.5$

If the x_1 variable is connected to the physical sense of the truss geometry, the position boundaries are: $x_{min} = 0.1 * h = 0.254m$ and $x_{max} = 2.25 * h = 5.72m$

The parameters are converted in the International System units. $\rho = 0.283lb/in^3 = 7833kg/m^3$; $h = 100 in = 2.54 m$; $P = 10^4lb = 44482.2N$; $E = 3 \cdot 10^7lb/in^2 = 2.07e11Pa$; $\sigma_0 = 2 \cdot 10^4lb/in^2 = 1.38e8Pa$; $A_{min} = 1in^2 = 6.5416e - 4m^2$.

3.2 Methodology

3.2.1 General explanation

The problem is defined with a class which contained four functions: two for the objectives and two for the constraints.

3.2.2 Penalty function

Pymoo library already contains constraint handling with constraint violation measures during the selection process. For the multi-objective problem, it uses the Pareto ranking. To study how the penalty strength affects convergence and feasibility, explicit penalty terms will be added.

$$F_{penalised} = f(x) + k * \sum (\max(0, g_i(x))^2) \quad (7)$$

With: k the penalty factor.

Considering this equation, a penalty will be applied when the maximum between 0 and g_i is g_i so the constraint is violated. This is realised with the following loop where the penalty factor is fixed to 10 billion. If the penalty factor is large the algorithm avoids infeasible solutions quickly. While, if the penalty factor is small the algorithm may explore infeasible regions longer.

3.3 Results

3.3.1 PSO simulation

Best solution

To obtain the best solution, the algorithm minimises the weight function and then the displacement function. Then it finds the best compromise to satisfy the objective functions following the constraint function.

Table 1: Summary of key solutions (PSO)

Solution type	x1 [normalised]	x2 [normalised]	Weight [kg]	Displacement [m]
Min weight	0.8245	0.5	16.6361	0.0084
Min displacement	1.8777	2.5	136.5345	0.0011
Compromise	1.2560	1.1667	48.0982	0.0029

All the possible values of the design variables are shown in the decision space diagram: each point represents a candidate solution. The colour show a gradient: dark colours for the lower objective values which are better for minimisation, and lighter colours for the higher objective values which are worse for minimisation.

Metrics

The Pareto front evolution across generations diagram shows that the first generations are far away from the Pareto front but through each time step the solutions converge. This

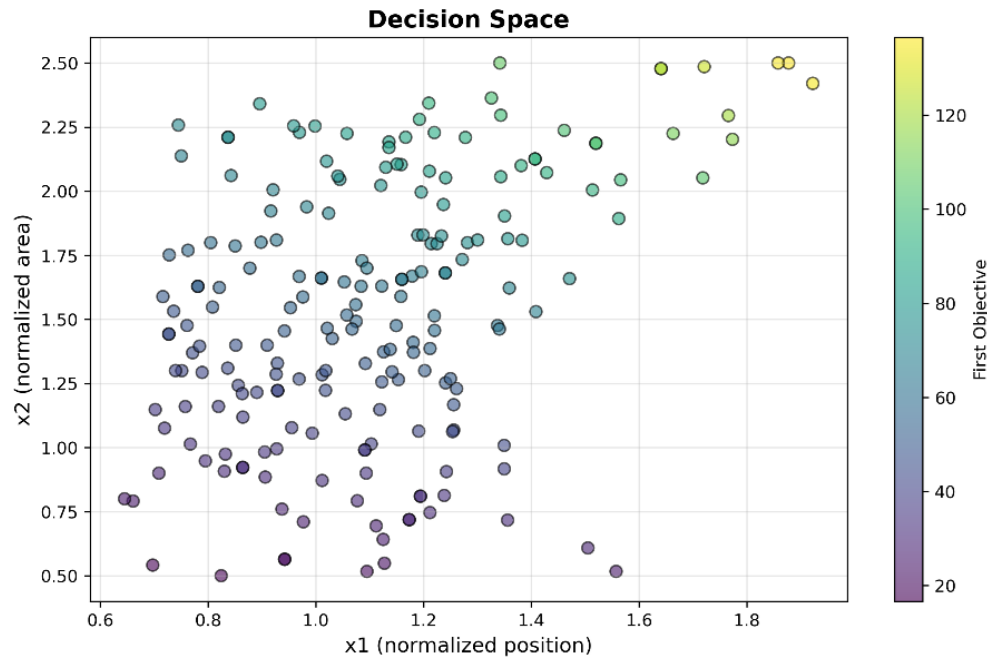


Figure 2: PSO Decision Space

is possible thanks to the penalty function which eliminates poor solutions. The visualisation helps identify regions of the decision space associated with optimal solutions.

The generation 199, or 200 time step, generates the Pareto front kept. The best solution is extracted from it.

The convergence analysis evaluates the efficiency of the algorithm. The following diagrams describe the evolution of the best values at each generation.

- **Best value First objective:** The weight is minimised over time with an improvement of the solution. The curve decreases over the generation, so the minimum is stable.
- **Best value Second objective:** The algorithm finds solutions with smaller solutions through the generations. The curve converges around an optimal displacement.
- **Normalised convergence:** The objectives are normalised for comparison. The algo-

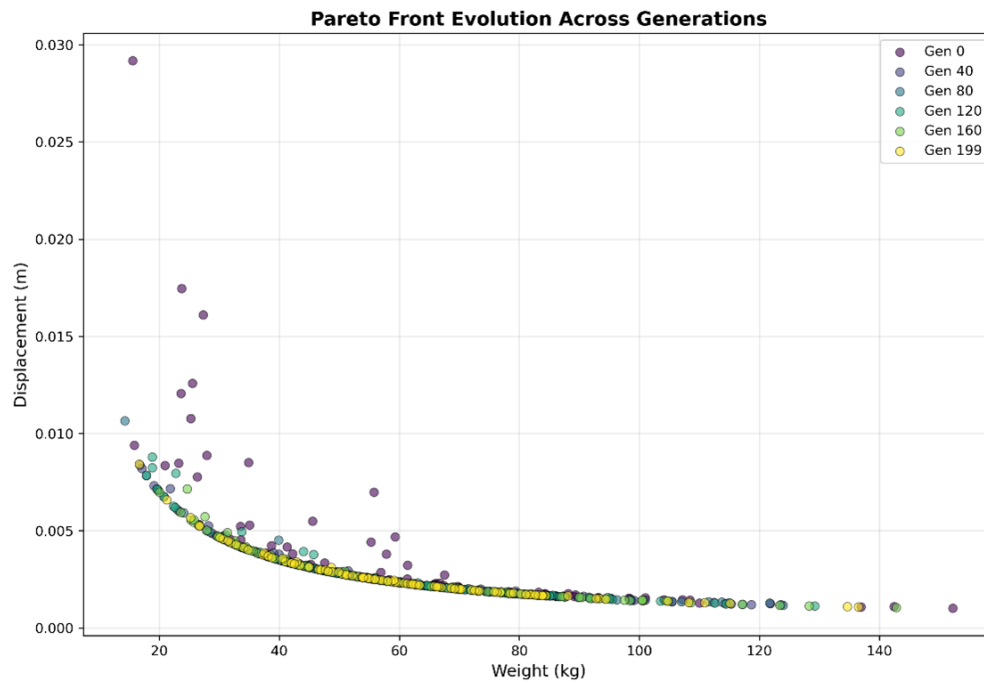


Figure 3: Pareto Front Evolution Across Generations

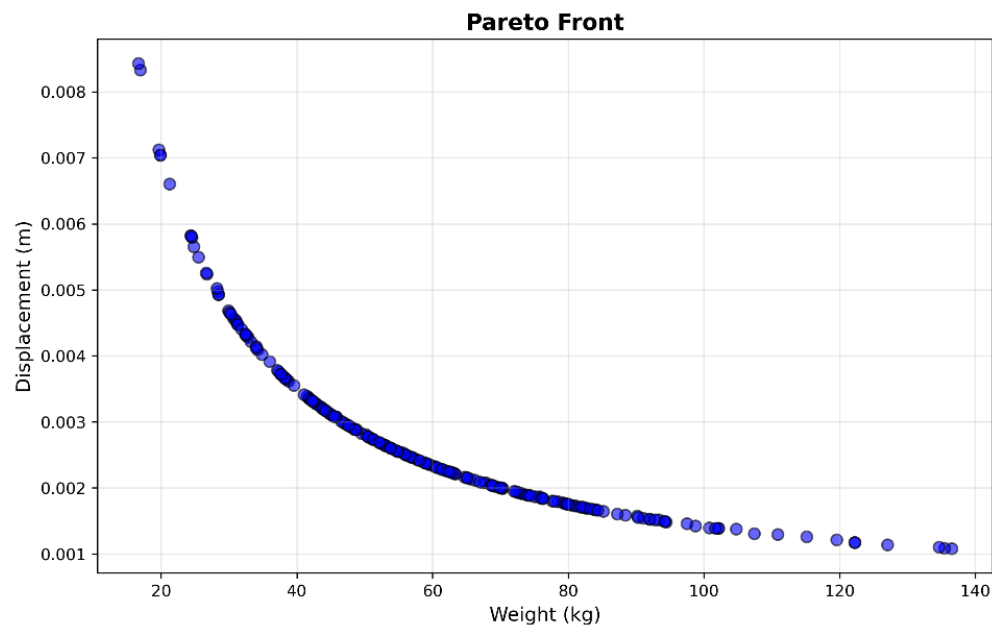


Figure 4: PSO Pareto Front

rithm is improving both objectives simultaneously since the curves decreases.

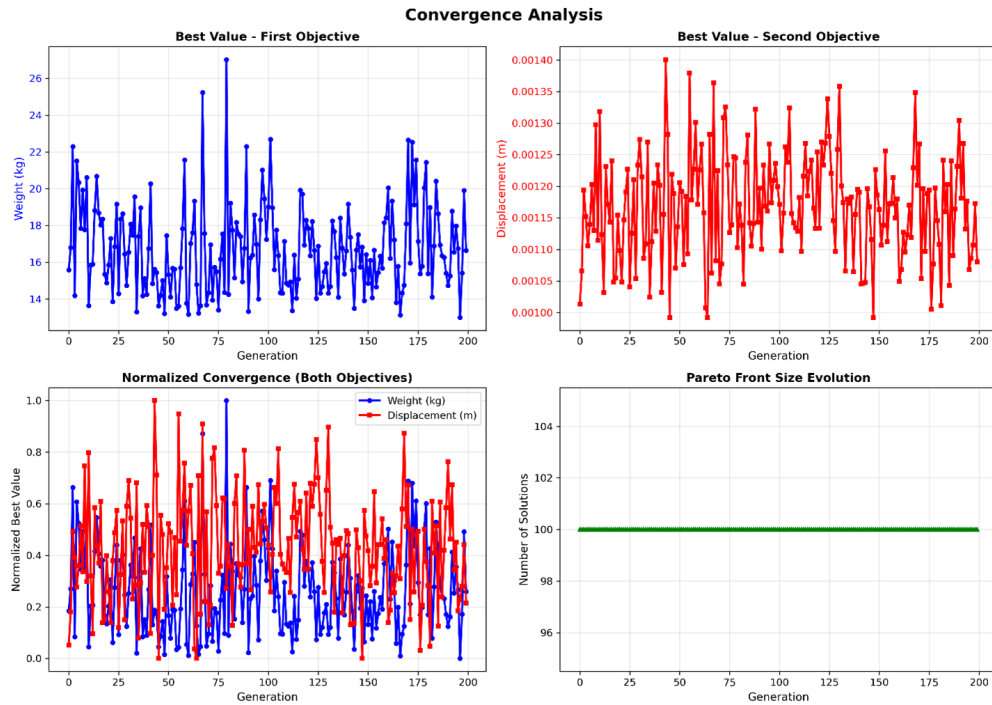


Figure 5: PSO Convergence Analysis

The Pareto front size evolution diagram shows the number of non-dominant solutions at each generation. The constant size of it means diversity is maintained throughout optimisation. This is a good indicator to tell that the algorithm didn't lose diversity and didn't converge prematurely.

3.3.2 SA simulation

Weight Scalarization Strategy The simulated_annealing program is based on a mono-objective algorithm. We have a scalarization approach for transforming the problem (bi-objective) into a function mono-objective. To generate the Pareto front, 200 runs are made with the weight uniformly distributed between 0.005 and 0.995. A scale of 10000 is applied

at the displacement for normalizing both objectives to comparable orders of magnitude. The stress constraints are enforced through a penalty method is generated by a penalty method in the program for the optimization with a final verification of the final solution. This double approach guaranteed that all the Pareto front respect the structural constraints.

Scalar Energy function :

$$E(x) = w_1 \cdot f_1(x) + w_2 \cdot (scale_f2 \times f_2(x)) + \sum_{i=1,2} 10^6 \cdot \max(0, g_i(x)) \quad (8)$$

where :

- w_1, w_2 : scalar weight with $w_1 + w_2 = 1$ and $w_1 \in [0.005, 0.995]$
- $f_1(x)$: structural weight (kg)
- $f_2(x)$: displacement (m)
- $scale_f2 = 10000$: the objective normalize factor
- $g_i(x) \leq 0$: Stress constraints (tension g_1 and compression g_2)

Pareto Front The Pareto front is the result of 200 runs of Simulated Annealing program and exhibits an optimal distribution of solutions representing the trade-off structural weight and node displacement. The valid solutions are comprised between: Weight: 14.39 kg to 213.64 kg Displacement : 0.848 mm to 8.382 mm The banana shape of the Pareto front saw the conflict between the structural optimization: reduce the weight and increase the flexibility and the displacement, otherwise if we want rigidity we need more material. We also saw no single solution minimizes in the same time both objectives.

Decision Space The decision space distribution revel a characteristic L shape. It is the result of the linear scalarization of the weight:

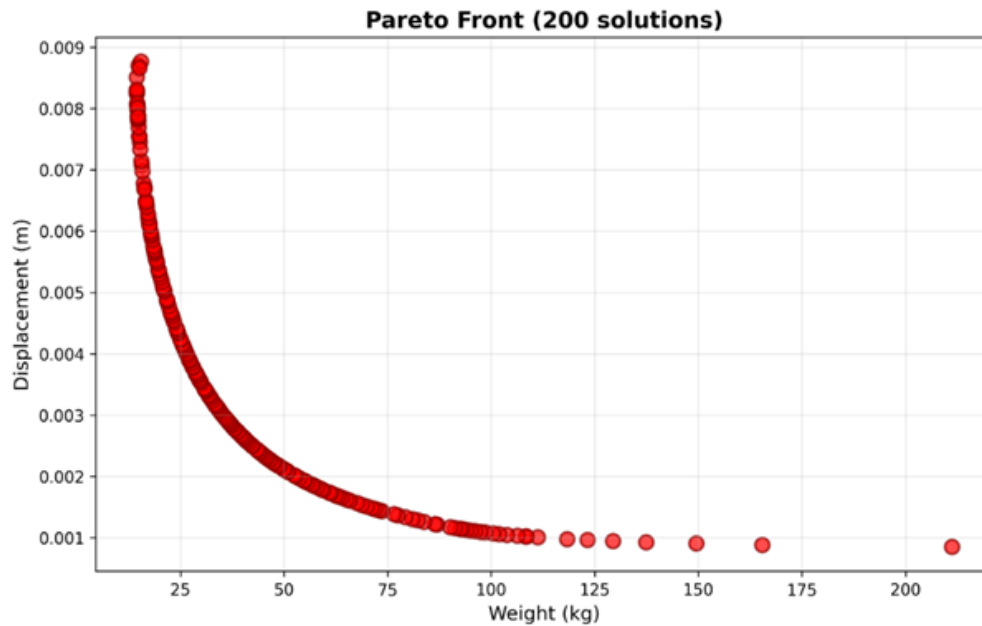


Figure 6: SA Pareto Front

- Region 1 ($x_2 \in [0.5, 1.0]$, x_1 variable): light solution for the weight minimization ($w_1 \approx 1.0$)
- Region 2 ($x_2 \in [2.0, 2.5]$, x_1 variable): Minimize the node displacement ($w_1 \approx 0.0$)
- Space of transition ($x_2 \in [1.0, 2.0]$) : compromise solution

Analyse de Convergence The convergence compartment of the 200 runs is represented on distinct plot on each objective:

- Best weight evolution: Weight decreased to 200 kg by 14,39 kg. Significant improvement result after the 50 first runs. Stable execution after 150 runs that saw a complete exploration of the light region.
- Best displacement evolution: Speed convergence in 20 runs to the minimal value of 0,848 mm. We have a parabolic augmentation and this logic we start by making the

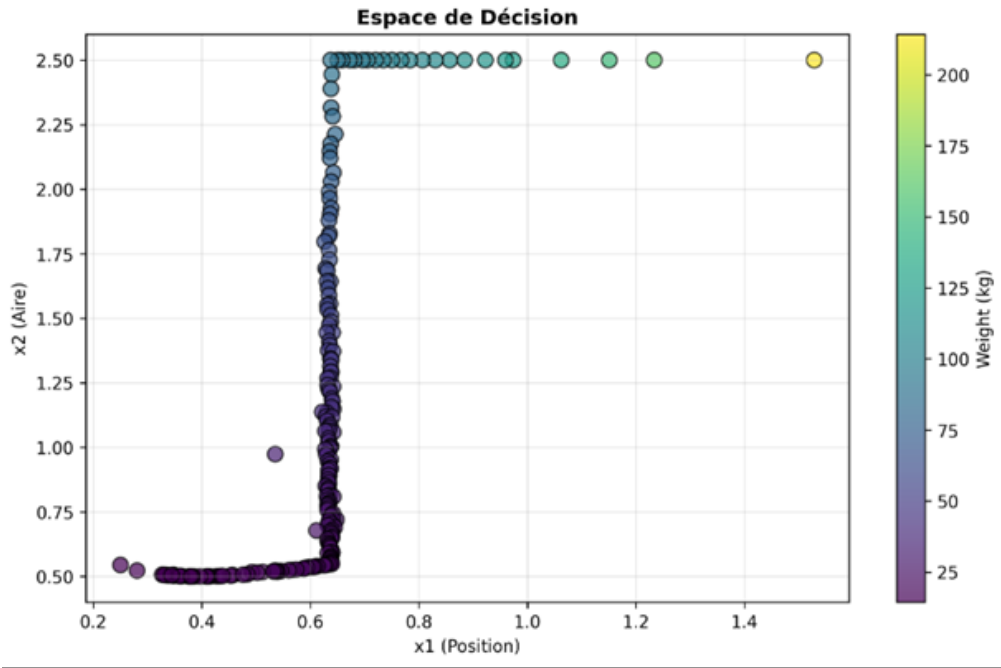


Figure 7: SA Decision Space

displacement optimization, so we have the best optimization at the beginning and the worst after that.

3.4 Discussion

Comparison between PSO and SA simulation:

Table 2: Comparison PSO vs SA

Algorithm	x1 [normalised]	x2 [normalised]	Weight [kg]	Displacement [m]
PSO	1.2560	1.1667	48.0982	0.0029
SA	(Compromise)	(Compromise)	(See Front)	(See Front)

(Note: SA produced a full Pareto front via scalarization, while MOPSO produced it directly. Both methods successfully mapped the trade-off curve.)

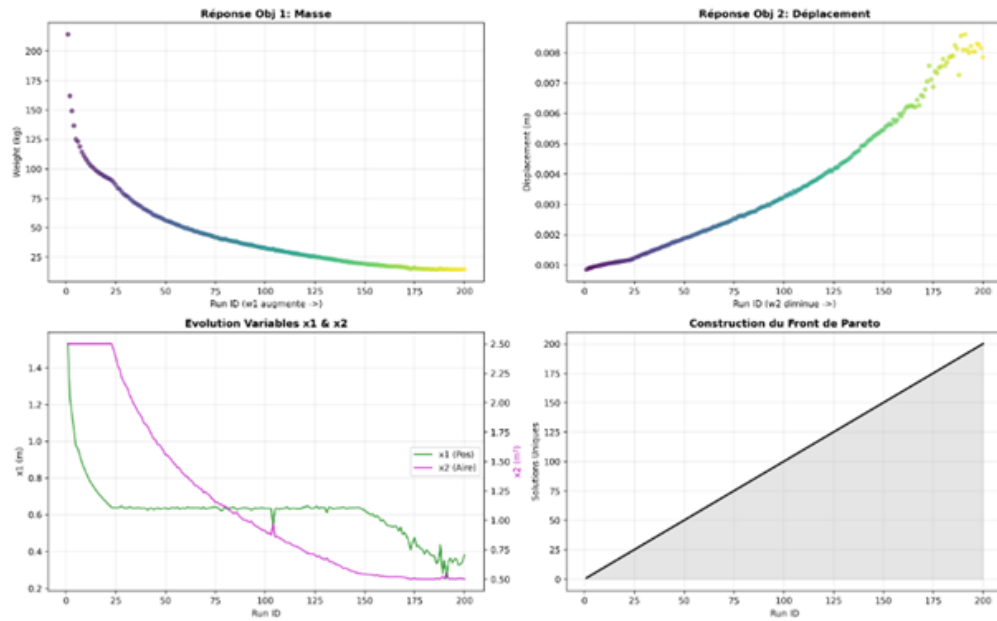


Figure 8: SA Convergence

4 Problem 2: Sensor Calibration

4.1 Problem description

The objective is to determine the Steinhart-Hart coefficients (A, B, C) that model the relationship between Resistance (R) and Temperature (T) for a 10K thermistor. Model Equation:

$$T_{model} = \frac{1}{A + B \ln(R) + C(\ln(R))^3} \quad (9)$$

Note: T is in Kelvin, R in Ohms.

4.2 Cost function

The optimisation goal is to minimise the Sum of Squared Errors (SSE) between the measured temperature and the model-predicted temperature:

$$SSE = \sum (T_{measured,i} - T_{model,i})^2 \quad (10)$$

4.3 Methodology

Simulated Annealing: A probabilistic technique for approximating the global optimum of a given function.

Particle Swarm Optimisation (PSO): A computational method that optimises a problem by iteratively trying to improve a candidate solution based on a given measure of quality. It solves a problem by maintaining a population of candidate solutions, here dubbed particles, and moving these particles around in the search space according to a simple mathematical formula that depends on the particles' positions and velocities.

4.4 Settings

Settings SA:

- Initial Temperature: 50.0
- Cooling Rate: 0.995
- Minimum Temperature: 1e-6

Setting PSO: Implementation utilizes the pymoo library (PSO algorithm).

- **Parameters:**

- Population size: 50
 - Generations: 500
 - Inertia weight (w): 0.7
 - Cognitive parameter (c_1): 1.5
 - Social parameter (c_2): 1.5
- max_velocity_rate: Limits the maximum velocity of particles.

4.5 Result

The optimal Steinhart-Hart coefficients found for the provided dataset are:

Table 3: Optimization Results Comparison (Steinhart-Hart)

Algorithm	A	B	C	Final SSE	Function Calls	Execution Time (s)
Simulated Annealing	1.114×10^{-3}	2.367×10^{-4}	7.761×10^{-8}	0.4323	3,537	0.099
PSO	1.127×10^{-3}	2.345×10^{-4}	8.625×10^{-8}	0.0089	25,000	0.678

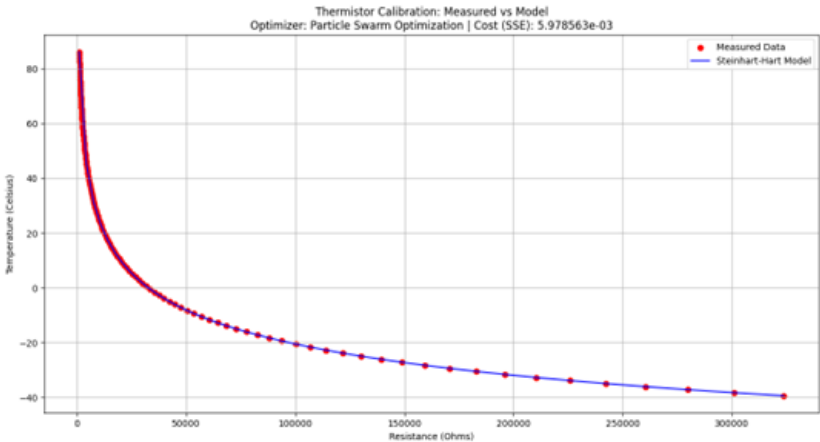
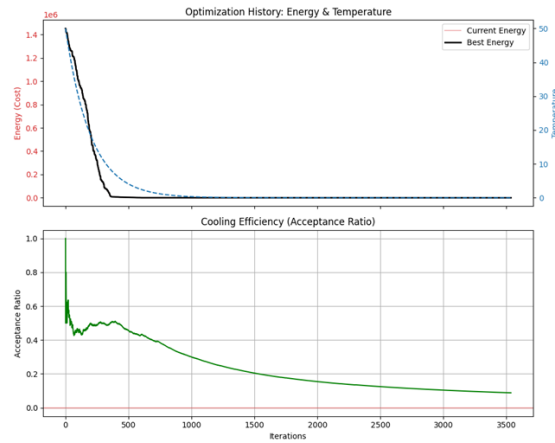
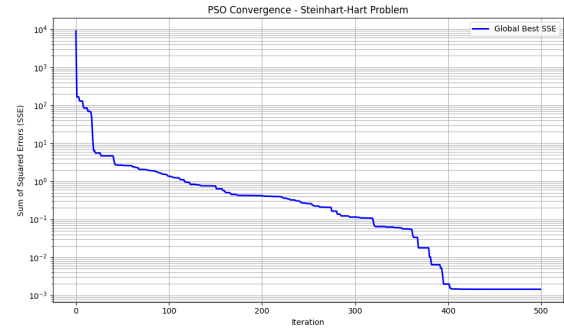


Figure 9: Steinhart-Hart Model Fit



(a) SA Convergence History



(b) PSO Convergence History

Figure 10: Convergence Comparison

4.6 Work discussion

Both algorithms successfully calibrated the thermistor model. PSO achieved a significantly lower final SSE (0.0089 versus 0.4323 for SA), indicating a more precise fit for this specific dataset and configuration. However, SA was roughly $6.8\times$ faster (0.099s vs 0.678s) and used significantly fewer function calls (3,537 vs 25,000). This illustrates the classic trade-off: PSO's population-based search provided a better global optimum at the cost of higher computational expense, while SA provided a "good enough" solution very quickly.

5 Problem 3: Gear Train Design

5.1 Problem description

The objective of this exercise is to choose a gear train design that achieves a specific transmission ratio. We would define the gear train with four gears with the numbers of teeth N_1, N_2, N_3, N_4 . The target transmission ratio is:

$$R = \frac{N_2 \cdot N_4}{N_1 \cdot N_3} \quad (11)$$

(Note: Standard formulation is usually $\frac{\text{product driven}}{\text{product drivers}}$ or similar. The equation in text is $R = (N_2 N_4)/(N_1 N_3)$ or $f(N) = (N_1 N_3)/(N_2 N_4)$. I will use the cost function definition for consistency: $Target \approx 1/6.931 \approx 0.144$).

Here, the target transmission ratio will be randomly chosen: $R_{target} = 1/6.931 \approx 0.1442793$. And each gear must have an integer number of teeth between 12 and 60. The problem is difficult because the search space is discrete and nonlinear, meaning classical optimization cannot be used directly. Therefore, two heuristic optimization algorithms will be tested:

- Particle Swarm Optimization (PSO)
- Simulated Annealing (SA)

The goal is to compare their performance and their behavior on this discrete optimization problem.

5.2 Cost function

The cost function measures the error between the achieved ratio and the target ratio:

$$f(N_1, N_2, N_3, N_4) = \left| \frac{N_1 \times N_3}{N_2 \times N_4} - R_{target} \right| \quad (12)$$

A good solution is one where this error approaches zero.

5.3 Sources

Course material on PSO and Simulated Annealing.

5.4 Methodology

Particle Swarm Optimization: It simulates a swarm of particles exploring the search space. Each particle represents a potential gear combination $[N_1, N_2, N_3, N_4]$. Particles move according to:

- Their own best-found solution
- The best solution found by the swarm

Positions are rounded to integers to satisfy the gear teeth constraint as we cant make gears with an odd number of teeth. We store the best error at each iteration to generate a convergence curve like we did for the SA.

Simulated Annealing: It explores the space by randomly perturbing the current solution. It accepts worse solutions with a probability depending on analogy to a temperature parameter:

- With high temperature goes a wide exploration

- Low temperature goes a convergence around a good solution

The algorithm stops when the temperature becomes very low. We record both the current energy and the best energy to produce a convergence curve.

5.5 Settings

PSO

- Population size: 40
- Iterations: 300
- $w = 0.7, C_1 = 1.5, C_2 = 1.5$
- Integer rounding of particle positions
- Recorded global best error per iteration (for convergence curve)

SA

- Initial temperature: 1000
- Cooling rate: 0.90
- Minimum temperature: 0.1
- Random seed: 42
- Recorded: Current energy / Best energy (for convergence curve)

5.6 Results

5.6.1 Results analysis

The optimal solution found by the Particle Swarm Optimization is:

PSO Results: PSO finds an extremely accurate gear ratio, with an error on the order of 10^{-5} , meaning it almost exactly matches the target ratio.

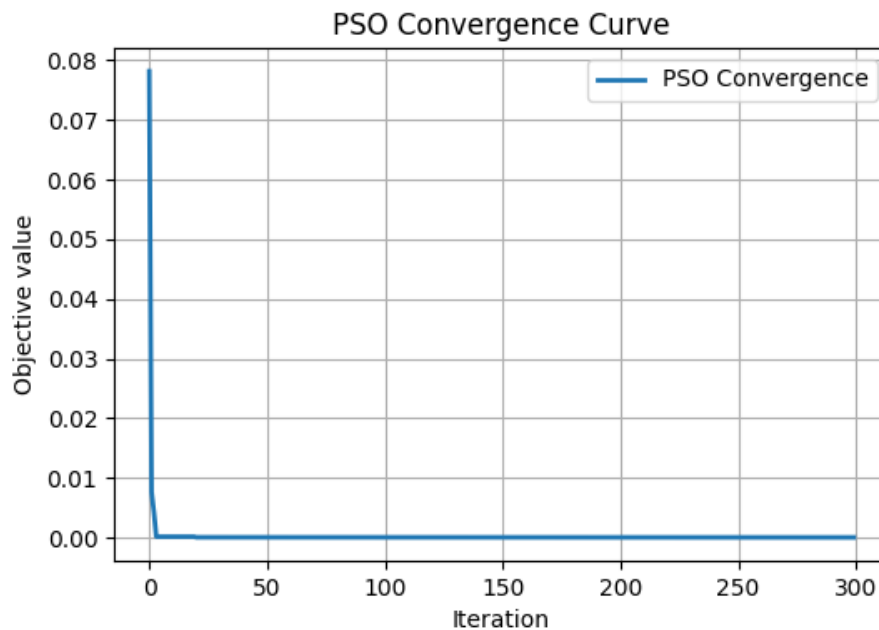


Figure 11: PSO Convergence curve

The graph shows that PSO converges extremely quickly within ≈ 10 iterations and then stabilizes at a very small error value.

And the result found by the Simulated Annealing (Solid Annealing) is:

SA Results: SA clearly fails on this problem. The final ratio is far from the target, with a very large error.

The plot shows:

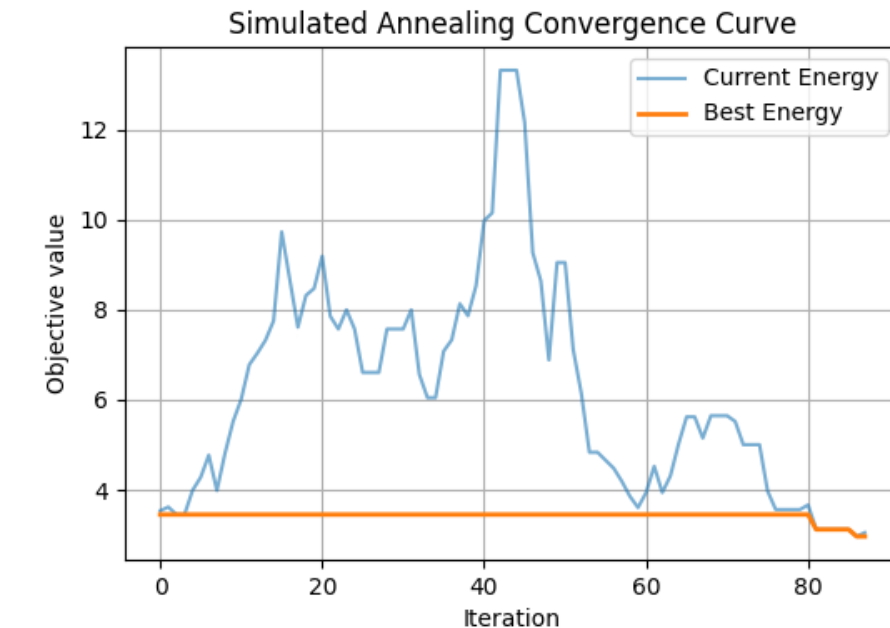


Figure 12: SA Convergence Curve

- Highly unstable exploration, current energy jumps up and down
- Best energy stays flat because the algorithm never discovers a good solution
- Cooling happens too fast to explore thoroughly

5.6.2 Explanations

Why PSO succeeds

- PSO is particularly effective in structured continuous search spaces, even with integer rounding.
- The particles rapidly converge to the correct ratio because the cost function is smooth.
- Global and personal best mechanisms guide the search efficiently.

Why SA fails

- The gear train problem is very sensitive: small changes in N_i produce large jumps in the ratio.
- SA explores randomly and relies on thermal acceptance, which:
 - Moves too chaotically
 - Does not exploit promising regions enough
 - Requires much slower cooling to perform well
- SA found a physically valid gear set but not even close to the desired ratio.

5.6.3 Conclusion

PSO is clearly the superior method for this task. It finds a nearly perfect solution extremely fast, while SA remains stuck far away from any optimal region. For a future study, SA could be improved by:

- Reducing the cooling rate from 0.99 to 0.90 for example
- Using a smarter neighbour generator
- Introducing integer-specific heuristics

But in its current form, SA is not suitable for this optimisation problem.

6 General Conclusion

The investigation into Particle Swarm Optimisation (PSO) and Simulated Annealing (SA) across three engineering domains yields the following conclusions:

- **PSO** demonstrated superior performance in Multi-Objective problems (Truss) and Discrete problems (Gear Train). Its population-based approach allows for better exploration of complex search spaces and effective handling of Pareto optimality.
- **SA** performed well in the continuous, single-objective parameter estimation (Sensor), providing accurate results very efficiently. However, it struggled with the discrete nature of the Gear Train problem without specific tuning of the neighbor generation.

6.1 Performance Comparison

This table summarises the computational performance of the algorithms.

Table 4: Algorithm Performance Metrics

Problem	Algorithm	Run Time (s)	Total Function Calls	Outcome
Two-Bar Truss	PSO (MOPSO)	5.833	20,000	Success (Pareto Front)
Sensor Calib.	SA	0.099	3,537	Success (SSE 0.43)
Gear Train	PSO	0.028	12,000	Success (Error < 10 ⁻⁴)
Gear Train	SA	0.0002	88	Failed (High Error)

Note: Function Calls for PSO = Population × Generations (e.g., 40 × 300 = 12,000 for Gear Train). SA Function Calls = Iterations.

6.2 Hyperparameter Sensitivity and Robustness

It is important to acknowledge that the results presented in this report are strictly dependent on the specific hyperparameters selected for each algorithm (e.g., cooling rate α for SA,

inertia weight w and social/cognitive parameters c_1, c_2 for PSO).

Stochastic optimization algorithms do not guarantee global convergence, and their performance is heavily influenced by these settings:

- **Simulated Annealing Sensitivity:** In Problem 3 (Gear Train), the SA algorithm failed to converge to the target ratio, resulting in a high error. This failure highlights the sensitivity of SA to the cooling schedule and perturbation step size. A cooling rate that is too fast causes the algorithm to become trapped in local minima, as observed in the Gear Train convergence plot. Conversely, a cooling rate that is too slow results in excessive computational cost. The failure in the discrete domain suggests that the specific neighbor generation method used was not sufficiently tuned for that specific landscape.
- **PSO Exploration vs. Exploitation:** The PSO performance relies on balancing exploration (controlled by w) and exploitation (controlled by c_1, c_2). While the chosen values ($w = 0.9, c_1 = c_2 = 2.0$) proved robust for the Truss problem, different values could lead to premature convergence or failure to refine the solution.
- **Penalty Factors:** As demonstrated in Section 3.2.1, constrained optimization requires careful tuning of penalty factors. While our experiment showed robustness at $k = 10^{10}$, setting this value too low in other contexts could lead to the acceptance of infeasible solutions, while setting it too high can distort the objective function landscape, making it difficult for the algorithm to approach the feasible boundary.

Therefore, a limitation of this study is that the comparison is valid only for the specific set of parameters tested. A more comprehensive study would require a sensitivity analysis or

an automated hyperparameter tuning process (such as a Grid Search) to identify the optimal configuration for each specific problem type.

7 References

- [1] A. Riaz. *Computational Optimisation Design Lecture 2*. Lecture presented at: Computational Optimisation Design. [cited 2025 Oct 21]. Available from: https://canvas.cranfield.ac.uk/courses/37635/pages/day-2?module_item_id=737718. 2025.
- [2] A. Riaz. *Computational Optimisation Design Lecture 3*. Lecture presented at: Computational Optimisation Design. [cited 2025 Oct 22]. Available from: https://canvas.cranfield.ac.uk/courses/37635/pages/day-3?module_item_id=737719. 2025.
- [3] J. Blank. *MOPSO-CD: Multi-Objective Particle Swarm Optimization with Crowding Distance*. Pymoo library. [cited 2025 Dec 9]. Available from: https://pymoo.org/algorithms/moo/mopso_cd.html. 2020.

8 Appendix: Python Code

8.1 Two-Bar Plane Truss Problem Definition

Listing 1: Truss Problem Definition

```
"""
Subject:Two-Bar_Plane_Truss_Problem
Source:https://www.sciencedirect.com/science/article/pii/
        S0898122111010406
@author:Clémence-Philomène_HINOT_&_Claude_AI
"""

# ***** IMPORTATION *****

import math
import numpy as np
from pymoo.core.problem import Problem

# ***** CLASS DEFINITION *****

class Two_Bar_Plane_Truss_Problem():

    # ----- INITIALIZATION -----

    def __init__(self):
```

constants

self.rho = 7833 *# unit: kg/m³*

self.h = 2.54 *# unit: m*

self.P = 44482.2 *# unit: N*

self.E = 2.07e11 *# unit: Pa*

self.sigma0 = 1.38e8 *# unit: Pa*

self.A_min = 6.4516e-4 *# unit: m²*

variables

self.pos = 0.0

self.area = 0.0

physical boundaries

self.pos_min = 0.1 * self.h

self.pos_max = 2.25 * self.h

self.area_min = 0.5 * self.A_min

self.area_max = 2.5 * self.A_min

————— CONSTITUTIVE EQUATIONS —————

def f1(self):

 x1 = self.pos/self.h *# need to use normalised x1 in this*
 function

```
        return 2 * self.rho * self.h * self.area * math.sqrt(1 +  
            pow(x1, 2))  
  
def f2(self):  
    x1 = self.pos/self.h  
    return (self.P * pow(self.h, 3) * pow((1 + pow(x1, 2)),  
        1.5)) / ( math.sqrt(1 + pow(x1, 4)) * 2 * math.sqrt(2)  
        * self.E * x1 * self.area)  
  
# ————— CONSTRAINT EQUATIONS —————  
  
# Stress constraint (tension)  
def g1(self):  
    x1 = self.pos/self.h  
    return (self.P * (1 + x1) * math.sqrt(1 + pow(x1, 2)) / (2  
        * math.sqrt(2) * x1 * self.area)) - self.sigma0  
  
# Stress constraint (compression)  
def g2(self):  
    x1 = self.pos/self.h  
    return (self.P * (1 - x1) * math.sqrt(1 + pow(x1, 2)) / (2  
        * math.sqrt(2) * x1 * self.area)) - self.sigma0
```

```
# ————— EVALUATION FUNCTION —————

def evaluate(self, x):
    """
    Evaluate objectives and constraints

    param x: [x1_normalized, x2_normalized]
    return: ([f1, f2], [g1, g2])
    """

    # physical position
    self.pos = x[0] * self.h

    # physical area
    self.area = x[1] * self.A_min

    # boundaries verifications
    if not (self.pos_min <= self.pos <= self.pos_max and self.
            area_min <= self.area <= self.area_max):
        raise ValueError(f'x values outside boundaries: {
            position={self.pos}, area={self.area}')

    # weight
```

```
f1 = self.f1()
```

```
# movement
```

```
f2 = self.f2()
```

```
#constraints
```

```
g1 = self.g1()
```

```
g2 = self.g2()
```

```
# results
```

```
return [f1, f2], [g1, g2]
```

```
# ***** PYMOO WRAPPER *****
```

```
class Two_Bar_Plane_Truss_Problem_Pymoo(Problem):
```

```
    """Pymoo-compatible wrapper for Two_Bar_Plane_Truss_Problem"""
```

```
    def __init__(self, penalty_factor=0):
```

```
        # Create truss problem instance
```

```
        self.truss = Two_Bar_Plane_Truss_Problem()
```

```
        # access to the penalty factor
```

```
        self.penalty_factor = penalty_factor
```

```
# Initialize pymoo Problem

super().__init__(
    n_var=2,                # x1, x2
    n_obj=2,                # f1, f2
    n_constr=2,             # g1, g2
    xl=np.array([0.1, 0.5]), # lower bounds
    xu=np.array([2.25, 2.5]) # upper bounds
)

def _evaluate(self, x, out, *args, **kwargs):
    """Evaluate multiple solutions"""
    n_solutions = x.shape[0]
    F = np.zeros((n_solutions, 2))
    G = np.zeros((n_solutions, 2))

    for i in range(n_solutions):
        try:
            objectives, constraints = self.truss.evaluate(x[i,
                                                            :])
            F[i, :] = objectives
            G[i, :] = constraints
        except ValueError:
```

```
# Penalty for invalid solutions

F[i, :] = [1e10, 1e10]
G[i, :] = [1e10, 1e10]

# output

out["F"] = F
out["G"] = G
```

8.2 PSO Simulation Script (Truss)

Listing 2: PSO Multi-Objective Simulation

```
"""
Subject: Problem_Optimisation_with_PSO_multi-objective_algorithm
@author: Clémence-Philomène_HINOT_&_Claude_AI
"""

# ***** IMPORTATION *****

import numpy as np
import matplotlib
matplotlib.use('Agg') # Non-interactive backend
import matplotlib.pyplot as plt
import csv
import os
```

```
# PSO algorithms

from Particle_Swarm_Optimization_multi_obj import ParticleSwarm

# problems

from two_bar_plane_truss_problem import
    Two_Bar_Plane_Truss_Problem_Pymoo

# ===== VISUALIZATION FUNCTIONS =====

def plot_pareto_front(result, save_path, obj_labels=None):
    """
    Plot the Pareto front for bi-objective optimization

    :param result: Optimization result from pymoo
    :param save_path: Path to save the figure
    :param obj_labels: List of [xlabel, ylabel] for objectives
    """
    if obj_labels is None:
        obj_labels = ['Objective_1', 'Objective_2']

    fig, ax = plt.subplots(figsize=(10, 6))
    ax.scatter(result.F[:, 0], result.F[:, 1], color='blue', s=50,
               alpha=0.6, edgecolors='black')
```

```
ax.set_xlabel(obj_labels[0], fontsize=12)
ax.set_ylabel(obj_labels[1], fontsize=12)
ax.set_title('Pareto_Front', fontsize=14, fontweight='bold')
ax.grid(True, alpha=0.3)
fig.savefig(save_path, dpi=300, bbox_inches='tight')
plt.close(fig)
print(f"Pareto_Front_saved_to_{save_path}")

def plot_decision_space(result, save_path, var_labels=None):
    """
    Plot the decision space colored by first objective

    :param result: Optimization result from pymoo
    :param save_path: Path to save the figure
    :param var_labels: List of [xlabel, ylabel] for decision
        variables
    """
    if var_labels is None:
        var_labels = ['Variable_1', 'Variable_2']

    fig, ax = plt.subplots(figsize=(10, 6))
    scatter = ax.scatter(result.X[:, 0], result.X[:, 1], c=result.
```

```
F[:, 0],

                                cmap='viridis', s=50, alpha=0.6,
                                edgecolors='black')
ax.set_xlabel(var_labels[0], fontsize=12)
ax.set_ylabel(var_labels[1], fontsize=12)
ax.set_title('Decision_Space', fontsize=14, fontweight='bold')
ax.grid(True, alpha=0.3)
fig.colorbar(scatter, label='First_Objective')
fig.savefig(save_path, dpi=300, bbox_inches='tight')
plt.close(fig)

print(f"Decision_space_saved_to_{save_path}")

def plot_convergence(result, save_path, obj_labels=None):
    """
    Plot_comprehensive_convergence_history

    :param result: Optimization_result_from_pymoo
    :param save_path: Path_to_save_the_figure
    :param obj_labels: List_of_objective_names
    """
    if obj_labels is None:
        obj_labels = ['Objective_1', 'Objective_2']
```

```
# Check if callback data exists

if not hasattr(result, 'callback') or result.callback is None:
    print ("ERROR: Convergence data not available - callback
           was not properly initialized ")
    return

data = result.callback.data

if "best_f1" not in data or len(data["best_f1"]) == 0:
    print ("ERROR: No convergence data recorded")
    return

generations = np.arange(len(data["best_f1"]))

# Create figure with subplots

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Convergence Analysis', fontsize=16, fontweight='
            bold')

# Plot 1: Best values for each objective

ax1 = axes[0, 0]

ax1.plot(generations, data["best_f1"], 'b-o', label=obj_labels
```

```
[0], linewidth=2, markersize=4)

ax1.set_xlabel('Generation', fontsize=11)
ax1.set_ylabel(obj_labels[0], fontsize=11, color='b')
ax1.tick_params(axis='y', labelcolor='b')
ax1.grid(True, alpha=0.3)
ax1.set_title('Best Value - First Objective', fontsize=12,
             fontweight='bold')

# Plot 2: Second objective
ax2 = axes[0, 1]
ax2.plot(generations, data["best_f2"], 'r-s', label=obj_labels
        [1], linewidth=2, markersize=4)
ax2.set_xlabel('Generation', fontsize=11)
ax2.set_ylabel(obj_labels[1], fontsize=11, color='r')
ax2.tick_params(axis='y', labelcolor='r')
ax2.grid(True, alpha=0.3)
ax2.set_title('Best Value - Second Objective', fontsize=12,
             fontweight='bold')

# Plot 3: Both objectives on same plot (normalized)
ax3 = axes[1, 0]
f1_norm = (data["best_f1"] - np.min(data["best_f1"])) / (np.
               max(data["best_f1"]) - np.min(data["best_f1"]) + 1e-10)
```

```
f2_norm = (data["best_f2"] - np.min(data["best_f2"])) / (np.
    max(data["best_f2"]) - np.min(data["best_f2"]) + 1e-10)
ax3.plot(generations, f1_norm, 'b-o', label=obj_labels[0],
    linewidth=2, markersize=4)
ax3.plot(generations, f2_norm, 'r-s', label=obj_labels[1],
    linewidth=2, markersize=4)
ax3.set_xlabel('Generation', fontsize=11)
ax3.set_ylabel('Normalized_Best_Value', fontsize=11)
ax3.legend(loc='best', fontsize=10)
ax3.grid(True, alpha=0.3)
ax3.set_title('Normalized_Convergence_(Both_Objectives)',
    fontsize=12, fontweight='bold')
```

Plot 4: Number of non-dominated solutions

```
ax4 = axes[1, 1]
ax4.plot(generations, data["n_nds"], 'g-^', linewidth=2,
    markersize=4)
ax4.set_xlabel('Generation', fontsize=11)
ax4.set_ylabel('Number_of_Solutions', fontsize=11)
ax4.grid(True, alpha=0.3)
ax4.set_title('Pareto_Front_Size_Evolution', fontsize=12,
    fontweight='bold')
```

```
fig.savefig(save_path, dpi=300, bbox_inches='tight')

plt.close(fig)

print(f"Convergence plot saved to {save_path}")

def plot_pareto_evolution(result, save_path, obj_labels=None,
                           generations_to_plot=None):
    """
    Plot evolution of Pareto front over generations

    :param result: Optimization result from pymoo
    :param save_path: Path to save the figure
    :param obj_labels: List of objective names
    :param generations_to_plot: List of generation indices to plot
        (default: evenly spaced)
    """
    if obj_labels is None:
        obj_labels = ['Objective_1', 'Objective_2']

    if not hasattr(result, 'callback') or result.callback is None:
        print("ERROR: Cannot plot Pareto evolution - callback data
              not available")

    return
```

```
data = result.callback.data

F_history = data["F"]

n_gens = len(F_history)


if generations_to_plot is None:
    # Plot 6 evenly spaced generations
    indices = [0, n_gens//5, 2*n_gens//5, 3*n_gens//5, 4*
               n_gens//5, n_gens-1]
    generations_to_plot = [i for i in indices if i < n_gens]


fig, ax = plt.subplots(figsize=(12, 8))
colors = plt.cm.viridis(np.linspace(0, 1, len(
    generations_to_plot)))


for idx, gen in enumerate(generations_to_plot):
    F = F_history[gen]
    ax.scatter(F[:, 0], F[:, 1], color=colors[idx], s=40,
               alpha=0.6,
               label=f'Gen_{gen}', edgecolors='black',
               linewidth=0.5)


ax.set_xlabel(obj_labels[0], fontsize=12)
ax.set_ylabel(obj_labels[1], fontsize=12)
```

```
ax.set_title('Pareto_Front_Evolution_Across_Generations',
             fontsize=14, fontweight='bold')
ax.legend(loc='best', fontsize=10)
ax.grid(True, alpha=0.3)
```

```
fig.savefig(save_path, dpi=300, bbox_inches='tight')
plt.close(fig)
print(f"Pareto_evolution_plot_saved_to_{save_path}")
```

```
# ===== EXPORT FUNCTIONS =====
```

```
def save_results_to_csv(result, save_path, var_names=None,
                        obj_names=None):
    """
```

```
    Save optimization results to CSV
```

```
    param result: Optimization result from pymoo
```

```
    param save_path: Path to save CSV file
```

```
    param var_names: List of variable names
```

```
    param obj_names: List of objective names
```

```
    """
```

```
    n_var = result.X.shape[1]
```

```
    n_obj = result.F.shape[1]
```

```
if var_names is None:
    var_names = [f'x{i+1}' for i in range(n_var)]
if obj_names is None:
    obj_names = [f'f{i+1}' for i in range(n_obj)]

with open(save_path, 'w', newline='') as f:
    writer = csv.writer(f)

    # Header
    header = var_names + obj_names
    if hasattr(result, 'G') and result.G is not None:
        n_constr = result.G.shape[1]
        header += [f'g{i+1}' for i in range(n_constr)]
    writer.writerow(header)

    # Data
    for i in range(len(result.F)):
        row = list(result.X[i, :]) + list(result.F[i, :])
        if hasattr(result, 'G') and result.G is not None:
            row += list(result.G[i, :])
        writer.writerow(row)
```

```
    print(f"Results saved to {save_path}")

def save_convergence_to_csv(result, save_path, obj_names=None):
    """
    Save convergence data to CSV

    :param result: Optimization result from pymoo
    :param save_path: Path to save CSV file
    :param obj_names: List of objective names
    """
    if not hasattr(result, 'callback') or result.callback is None:
        print("WARNING: No convergence data to save")
        return

    data = result.callback.data

    if obj_names is None:
        obj_names = ['Objective_1', 'Objective_2']

    with open(save_path, 'w', newline='') as f:
        writer = csv.writer(f)

        # Header
```

```
header = [ 'Generation ', f'Best_{obj_names[0]} ', f'Best_{
            obj_names[1]} ', 'Pareto_Front_Size' ]
writer.writerow(header)

# Data

for i in range(len(data["best_f1"])):
    row = [i, data["best_f1"][i], data["best_f2"][i], data
           ["n_nds"][i]]
    writer.writerow(row)

print(f"Convergence_data_saved_to_{save_path}")

def save_summary(result, save_path, var_names=None, obj_names=None
):
    """
    Save_summary_of_best_solutions_to_CSV

    param result: Optimization_result_from_pymoo
    param save_path: Path_to_save_summary_CSV
    param var_names: List_of_variable_names
    param obj_names: List_of_objective_names
    """
    n_var = result.X.shape[1]
```

```
n_obj = result.F.shape[1]

if var_names is None:
    var_names = [f'x{i+1}' for i in range(n_var)]
if obj_names is None:
    obj_names = [f'f{i+1}' for i in range(n_obj)]

# Find key solutions
min_indices = [np.argmin(result.F[:, i]) for i in range(n_obj)]
]
middle_idx = len(result.F) // 2

with open(save_path, 'w', newline='') as f:
    writer = csv.writer(f)

    # Header
    writer.writerow(['Summary of Key Solutions'])
    writer.writerow(['Solution Type'] + var_names + obj_names)

    # Min solutions for each objective
    for i, idx in enumerate(min_indices):
        row = [f'Min_{obj_names[i]}'] + list(result.X[idx, :])
            + list(result.F[idx, :])
```

```
writer.writerow(row)

# Compromise solution
row = [ 'Compromise' ] + list(result.X[middle_idx, :]) +
      list(result.F[middle_idx, :])
writer.writerow(row)

print(f"Summary_saved_to_{save_path}")

# ===== MAIN OPTIMIZATION RUNNER =====

def run_optimization(problem, pso_params, output_dir='results',
                    var_names=None, obj_names=None,
                    var_labels=None, obj_labels=None):
    """
    Run PSO optimization on a given problem

    param problem: Pymoo Problem instance
    param pso_params: Dictionary of PSO parameters
    param output_dir: Directory to save results
    param var_names: List of variable names for CSV
    param obj_names: List of objective names for CSV
```

```
####: param_var_labels: List of variable labels for plots
####: param_obj_labels: List of objective labels for plots
####: return: Optimization result
#### " " "
```

```
print("="*70)
print("MULTI-OBJECTIVE OPTIMIZATION USING PSO")
print("="*70)
```

```
# Create output directory
```

```
os.makedirs(output_dir, exist_ok=True)
```

```
# Initialize PSO
```

```
print("\n[1/3] Initializing PSO optimizer...")
pso = ParticleSwarm(problem, pso_params)
print("PSO initialized")
```

```
# Run optimization
```

```
print("\n[2/3] Running optimization...")
import time
start_time = time.time()
result = pso.PSO_multi_objective(Verbose=True)
end_time = time.time()
```

```
print(f"Execution_Time:{end_time-start_time:.4f}seconds")  
print(f"Optimization_complete_Found{len(result.F)}Pareto_  
    solutions")
```

```
# Save results
```

```
print("\n[3/3]Saving results...")
```

```
# CSV files
```

```
csv_path = os.path.join(output_dir, 'pareto_solutions.csv')
```

```
save_results_to_csv(result, csv_path, var_names, obj_names)
```

```
summary_path = os.path.join(output_dir, 'summary.csv')
```

```
save_summary(result, summary_path, var_names, obj_names)
```

```
convergence_csv_path = os.path.join(output_dir, '  
    convergence_data.csv')
```

```
save_convergence_to_csv(result, convergence_csv_path,  
    obj_names)
```

```
# Plots
```

```
pareto_path = os.path.join(output_dir, 'pareto_front.png')
```

```
plot_pareto_front(result, pareto_path, obj_labels)
```

```
decision_path = os.path.join(output_dir, 'decision_space.png')
plot_decision_space(result, decision_path, var_labels)
```

```
convergence_path = os.path.join(output_dir, 'convergence.png')
plot_convergence(result, convergence_path, obj_labels)
```

```
evolution_path = os.path.join(output_dir, 'pareto_evolution.
    png')
plot_pareto_evolution(result, evolution_path, obj_labels)
```

```
print("\n" + "="*70)
print("OPTIMIZATION_COMPLETE!")
print("="*70)
```

```
return result
```

```
# ===== PROBLEM OPTIMISATION =====
```

```
if __name__ == "__main__":
```

```
    from two_bar_plane_truss_problem import
```

```
        Two_Bar_Plane_Truss_Problem_Pymoo
```

```
# Create problem
```

```
problem = Two_Bar_Plane_Truss_Problem_Pymoo()
```

```
# Configure PSO parameters
```

```
pso_params = {  
    'pop_size': 100,  
    'w': 0.9,  
    'c1': 2.0,  
    'c2': 2.0,  
    'max_velocity_rate': 0.2,  
    'n_gen': 200,  
    'seed': 42  
}
```

```
# Define labels for visualization and export
```

```
var_names = ['x1', 'x2']
```

```
obj_names = ['Weight', 'Displacement']
```

```
var_labels = ['x1_(normalized_position)', 'x2_(normalized_area  
    )']
```

```
obj_labels = ['Weight_(kg)', 'Displacement_(m)']
```

```
# Run optimization
```

```
result = run_optimization(  

```

```
        problem=problem ,  
        pso_params=pso_params ,  
        output_dir='results_truss' ,  
        var_names=var_names ,  
        obj_names=obj_names ,  
        var_labels=var_labels ,  
        obj_labels=obj_labels  
    )
```

8.3 Steinhart-Hart Problem Definition

Listing 3: Steinhart-Hart Problem

```
#Author: John.H, Gemini Google AI  
#Date: 2025-12-03  
#Version: 1.0  
#Description: Steinhart-Hart Problem
```

```
import math  
import random  
import csv  
from problem_interface import Problem  
  
class SteinhartHartProblem(Problem):  
    def __init__(self, data_file):
```

```
self.data = []

self._load_data(data_file)


def _load_data(self, filename):
    """Loads thermistor data from CSV."""
    with open(filename, 'r') as f:
        reader = csv.DictReader(f)
        for row in reader:
            # Store R and T (in Kelvin)
            #  $T_K = T_C + 273.15$ 
            r = float(row['Resistance_Ohms'])
            t_c = float(row['Temperature_C'])
            t_k = t_c + 273.15
            self.data.append({'R': r, 'T_K': t_k})


def generate_initial_solution(self):
    """
    Returns a random starting state [A, B, C].
    Typical values:  $A \sim 1e-3$ ,  $B \sim 1e-4$ ,  $C \sim 1e-7$ 
    """
    return [
        random.uniform(0.0005, 0.002),    # A
        random.uniform(0.0001, 0.0005),   # B
```

```
        random.uniform(1e-8, 1e-6)        # C
    ]

    def evaluate(self, solution):
        """
        Calculates SSE: sum((T_measured - T_model)^2)
        Model: 1/T = A + B * ln(R) + C * (ln(R))^3
        => T_model = 1 / (A + B * ln(R) + C * (ln(R))^3)
        """
        A, B, C = solution

        sse = 0.0

        for point in self.data:
            R = point['R']
            T_measured = point['T_K']

            try:
                ln_R = math.log(R)
                denom = A + B * ln_R + C * (ln_R ** 3)

                if denom == 0:
                    return float('inf')
```

```
T_model = 1.0 / denom

error = T_measured - T_model
sse += error ** 2
except (ValueError, ZeroDivisionError):
    return float('inf')

return sse

def get_neighbor(self, solution):
    """
    Perturbs [A, B, C] slightly.
    """
    A, B, C = solution

    # Perturbation factors tailored to the scale of parameters
    new_A = A + random.gauss(0, 1e-5)
    new_B = B + random.gauss(0, 1e-6)
    new_C = C + random.gauss(0, 1e-9)

    return [new_A, new_B, new_C]

def plot_model(self, solution, save_path=None, title=None):
```

```
"""
Plots the measured data vs the model curve.
"""

import matplotlib.pyplot as plt
import numpy as np

A, B, C = solution

# Extract data
R_data = [d['R'] for d in self.data]
T_data = [d['T_K'] - 273.15 for d in self.data] # Convert
back to Celsius for plotting

# Generate smooth curve
min_R = min(R_data)
max_R = max(R_data)
R_smooth = np.linspace(min_R, max_R, 100)
T_smooth = []

for r in R_smooth:
    try:
        ln_R = math.log(r)
        denom = A + B * ln_R + C * (ln_R ** 3)
```

```
        if denom != 0:
            t_k = 1.0 / denom
            T_smooth.append(t_k - 273.15)
        else:
            T_smooth.append(None)
    except ValueError:
        T_smooth.append(None)

plt.figure(figsize=(10, 6))
plt.scatter(R_data, T_data, color='red', label='Measured_
Data')
plt.plot(R_smooth, T_smooth, color='blue', label='
Steinhart-Hart_Model')

plt.xlabel('Resistance_(Ohms)')
plt.ylabel('Temperature_(Celsius)')
if title:
    plt.title(title)
else:
    plt.title('Thermistor_Calibration:_Measured_vs_Model')
plt.legend()
plt.grid(True)
```

```

    if save_path:
        plt.savefig(save_path)
        print(f"Plot saved to {save_path}")

plt.show()

```

8.4 Simulated Annealing Class

Listing 4: Simulated Annealing Implementation

```

# Objective: Simulated Annealing Optimization
# Author: John Hoarau, Gemini Google AI
# Date: 2025-12-03

```

```

import math
import random
import matplotlib.pyplot as plt
import csv

class SimulatedAnnealing:
    def __init__(self, problem, initial_temp=1000, cooling_rate
                =0.9, min_temp=0.1, seed=None):
        """
        :param seed: Integer to fix randomness for reproducibility
        """

```

```
        """
        self.problem = problem
        self.initial_temp = initial_temp
        self.temp = initial_temp
        self.cooling_rate = cooling_rate
        self.min_temp = min_temp

    # 1. Set the Seed
    if seed is not None:
        random.seed(seed)
        print(f"Random_Seed_set_to:{seed}")

    # Data Logging
    self.history = {
        "iteration": [],
        "temperature": [],
        "energy": [],
        "best_energy": [],
        "accepted": [] # 1 if accepted, 0 if rejected
    }
    self.best_solution = None

    def _update_temperature(self):
```

```

"""
Implements the Cooling Schedule.
Currently: Geometric cooling.
"""

self.temp *= self.cooling_rate

def solve(self, verbose=True):
    current_solution = self.problem.generate_initial_solution
    ()
    current_energy = self.problem.evaluate(current_solution)

    best_solution = current_solution
    best_energy = current_energy

    iteration = 0

    print("——Starting Optimization——")

    while self.temp > self.min_temp:
        # Create neighbor
        new_solution = self.problem.get_neighbor(
            current_solution)
        new_energy = self.problem.evaluate(new_solution)

```

```
energy_diff = new_energy - current_energy
accepted = False

# Acceptance Logic
if new_energy < current_energy:
    accepted = True
else:
    probability = math.exp(-energy_diff / self.temp)
    if random.random() < probability:
        accepted = True

# Update State
if accepted:
    current_solution = new_solution
    current_energy = new_energy
    if current_energy < best_energy:
        best_energy = current_energy
        best_solution = current_solution

# Store best solution in instance
self.best_solution = best_solution
```

```
# 2. Collect Metrics

self.history["iteration"].append(iteration)
self.history["temperature"].append(self.temp)
self.history["energy"].append(current_energy)
self.history["best_energy"].append(best_energy)
self.history["accepted"].append(1 if accepted else 0)


# Cool down

self._update_temperature()

iteration += 1


print(f"——_Finished_in_{iteration}_iterations——")
print(f"Final_Energy:_{{best_energy:.4f}}")


# Calculate final Acceptance Ratio

total_accepted = sum(self.history["accepted"])
ratio = total_accepted / iteration
print(f"Global_Acceptance_Ratio:_{{ratio:.2%}}")


return best_solution


def plot_statistics(self, save_path=None):
```

```
"""
Generates 2 graphs:
1. Energy & Temperature over time.
2. Acceptance Ratio over time.
"""

iterations = self.history["iteration"]

# Setup the plot
fig, (ax1, ax3) = plt.subplots(2, 1, figsize=(10, 8),
                               sharex=True)

# — Graph 1: Energy vs Temperature —
color = 'tab:red'
ax1.set_ylabel('Energy (Cost)', color=color)
ax1.plot(iterations, self.history["energy"], color=color,
         alpha=0.3, label="Current Energy")
ax1.plot(iterations, self.history["best_energy"], color='
         black', linewidth=2, label="Best Energy")
ax1.tick_params(axis='y', labelcolor=color)
ax1.legend(loc='upper right')

# Create a second y-axis for Temperature sharing the same
x-axis
```

```
ax2 = ax1.twinx()
color = 'tab:blue'
ax2.set_ylabel('Temperature', color=color)
ax2.plot(iterations, self.history["temperature"], color=
        color, linestyle='—', label="Temperature")
ax2.tick_params(axis='y', labelcolor=color)
ax1.set_title('Optimization_History: Energy & Temperature'
        )

# — Graph 2: Cumulative Acceptance Ratio —
# Calculate running average of acceptance
cumulative_acceptance = []
running_sum = 0
for i, val in enumerate(self.history["accepted"]):
    running_sum += val
    cumulative_acceptance.append(running_sum / (i + 1))

ax3.set_xlabel('Iterations')
ax3.set_ylabel('Acceptance_Ratio')
ax3.plot(iterations, cumulative_acceptance, color='green',
        label="Cumulative_Acceptance_Ratio")
ax3.axhline(y=0.0, color='r', linestyle='—', alpha=0.3)
ax3.set_title('Cooling_Efficiency_(Acceptance_Ratio)')
```

```
ax3.grid(True)

plt.tight_layout()

if save_path:
    plt.savefig(save_path)
    print(f"Plot saved to {save_path}")

plt.show()

def plot_best_solution(self, save_path=None):
    """
    Delegates plotting to the problem class if available.
    """
    if hasattr(self.problem, 'plot_solution') and self.
        best_solution is not None:
        self.problem.plot_solution(self.best_solution,
            save_path)
    elif hasattr(self.problem, 'plot_model') and self.
        best_solution is not None:
        self.problem.plot_model(self.best_solution, save_path
        )
```

```
def plot_model(self, solution, save_path=None, title=None):
    if hasattr(self.problem, 'plot_model'):
        self.problem.plot_model(solution, save_path=save_path,
                                title=title)

def save_to_csv(self, filename="annealing_results.csv"):
    """
    Saves the history of the run to a CSV file.
    Format: iteration, temperature, energy, best_energy,
    accepted
    """

    # 1. Get the column names (headers)
    keys = list(self.history.keys())

    # 2. Get the data columns
    values = list(self.history.values())

    # 3. Zip them together (this turns columns into rows)
    rows = zip(*values)

    try:
        with open(filename, 'w', newline='') as f:
```

```
writer = csv.writer(f)

# Write the header
writer.writerow(keys)

# Write the data
writer.writerows(rows)

print(f"→Data successfully saved to '{filename}'")

except IOError as e:
    print(f"Error saving file: {e}")
```

8.5 Gear Train Problem and Runner

Listing 5: Gear Train Problem

```
# Objective: Define the Gear Train design optimization problem
# Author: Chenonceau Group, ChatGPT
# Date: 2025-12-10

import math
import random
import matplotlib.pyplot as plt

def gear_train_objective(x):
```

```

"""
Objective function for the gear train optimization.

x: iterable -> [N1, N2, N3, N4]
The values may be floats; we convert them to integers
because gear teeth must be whole numbers.

The function computes the gear ratio:
ratio = (N1 * N3) / (N2 * N4)

and returns the absolute error with respect to
a fixed target ratio 1 / 6.931.
"""

# Convert all values to integer gear tooth counts
N1, N2, N3, N4 = [int(round(v)) for v in x]

# Target gear ratio (fixed value)
target_ratio = 1.0 / 6.931

# Gear ratio produced by this gear train
ratio = (N1 * N3) / (N2 * N4)

```

```
# Objective: absolute error between achieved and target ratio  
return abs(ratio - target_ratio)
```

```
# Bounds for each gear tooth number (min, max)  
# Used by both PSO and Simulated Annealing  
bounds = [(12, 60), (12, 60), (12, 60), (12, 60)]
```

```
class GearTrainProblem:  
    """  
    Problem wrapper for the SimulatedAnnealing class.  
  
    This class provides the methods expected by SimulatedAnnealing  
    :  
    - generate_initial_solution()  
    - evaluate(solution)  
    - get_neighbor(solution)  
    - plot_model(solution, save_path=None)  
    """  
  
    def __init__(self):  
        self.bounds = bounds
```

```
self.target_ratio = 1.0 / 6.931

def generate_initial_solution(self):
    """
    Generate a random initial solution within the bounds.
    """
    return [random.uniform(b[0], b[1]) for b in self.bounds]

def evaluate(self, solution):
    """
    Evaluate a solution using the same objective function as
    for PSO.
    """
    return gear_train_objective(solution)

def get_neighbor(self, solution, step_ratio=0.1):
    """
    Generate a neighbor solution by changing one variable with
    a small random step.
    """
    new = list(solution)
    i = random.randrange(len(new))
    low, high = self.bounds[i]
```

```
step = (high - low) * step_ratio

new[i] = new[i] + random.uniform(-step, step)
new[i] = max(low, min(new[i], high))

return new
```

Listing 6: Gear Train Runner

```
# Objective: Execute optimization for Gear Train problem (PSO vs
SA)
# Author: Chenonceau Group, ChatGPT
# Date: 2025-12-10
```

```
from Gear_Train_Problem import gear_train_objective, bounds,
    GearTrainProblem
from Gear_Train_PSO import PSO
from simulated_annealing import SimulatedAnnealing
import time

def print_solution(label, solution, score):
    """
    Utility function to display a gear train solution and its
    error.
    """
```

```
N1, N2, N3, N4 = [int(round(v)) for v in solution]

achieved_ratio = (N1 * N3) / (N2 * N4)
target_ratio = 1.0 / 6.931
abs_error = abs(achieved_ratio - target_ratio)

print(f"\n===={label}====")
print("Best_gear_combination:", [N1, N2, N3, N4])
print("Achieved_ratio:", achieved_ratio)
print("Target_ratio:", target_ratio)
print("Absolute_error:", abs_error)
print("\n")

if __name__ == "__main__":

    # 1) Optimize with PSO
    start_pso = time.time()
    pso = PSO(
        objective_function=gear_train_objective,
        bounds=bounds,
        num_particles=40,
        max_iter=300,
```

```
w=0.7,
c1=1.5,
c2=1.5
)

best_pso, best_pso_score = pso.run()
end_pso = time.time()
pso_time = end_pso - start_pso

print(f"——_PSO_Finished_in_{pso.max_iter}_iterations_——")
print(f"PSO_Time:_{pso_time:.4f}_seconds")
print(f"Final_PSO_Energy:_{best_pso_score:.4f}")
print_solution("PSO_RESULT", best_pso, best_pso_score)

# 2) Optimize with Simulated Annealing

start_sa = time.time()
problem_sa = GearTrainProblem()

sa = SimulatedAnnealing(
    problem=problem_sa,
    initial_temp=1000,
    cooling_rate=0.9,
    min_temp=0.1,
```

```
        seed=42
    )

    best_sa = sa.solve(verbose=True)
    best_sa_score = problem_sa.evaluate(best_sa)
    end_sa = time.time()
    sa_time = end_sa - start_sa
    print(f"SA_Time: {sa_time:.4 f} seconds")

    print_solution("SIMULATED_ANNEALING_RESULT", best_sa,
                   best_sa_score)
```